

Curso de Automatización con Java, Selenium, JUnit y Gherkin

Clase 2 – Repaso Java II

COLEGIO
UNIVERSITARIO



CCC
Centro de Capacitación
Continua

Temario

- ◆ OOP aplicado a automatización (clases, objetos, herencia, polimorfismo)
- ◆ Colecciones y expresiones lambda

 Práctica: listas, mapas y filtrado de datos

Introducción a la OOP aplicada a automatización

Conceptos base:

- La OOP permite modelar el mundo real en código.
- En automatización, usamos clases para representar **páginas**, **componentes** o **flujos**.
- Principales pilares:
 - **Encapsulamiento**: proteger los datos.
 - **Herencia**: reutilizar código.
 - **Polimorfismo**: comportamientos distintos con una misma interfaz.
 - **Abstracción**: mostrar solo lo esencial.

```
public class PaginaBase {  
    protected WebDriver driver;  
  
    public PaginaBase(WebDriver driver) {  
        this.driver = driver;  
    }  
  
    public void navegarA(String url) {  
        driver.get(url);  
    }  
}
```

Clases y Objetos

```
public class Usuario {  
    String nombre;  
    String email;  
  
    public Usuario(String nombre, String email) {  
        this.nombre = nombre;  
        this.email = email;  
    }  
  
    public void mostrarDatos() {  
        System.out.println("Nombre: " + nombre);  
        System.out.println("Email: " + email);  
    }  
}  
  
// Uso  
Usuario u = new Usuario("Ana", "ana@test.com");  
u.mostrarDatos();
```

Aplicación en Selenium:

Cada **página** puede ser una clase con sus **elementos y acciones** encapsulados.

Encapsulamiento y Modificadores de acceso

Concepto:

- Proteger los datos internos de una clase.
- Se logra usando `private` y métodos públicos `getter/setter`.

```
public class Cuenta {  
    private double saldo;  
  
    public void depositar(double monto) {  
        if (monto > 0) saldo += monto;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
}
```

Herencia y Polimorfismo

Herencia:

Permite que una clase (hija) reutilice código de otra (padre).

```
public class Empleado {  
    public void trabajar() {  
        System.out.println("Empleado trabajando");  
    }  
}  
  
public class Tester extends Empleado {  
    public void trabajar() {  
        System.out.println("Tester ejecutando pruebas");  
    }  
}  
  
// Uso:  
Empleado e1 = new Tester();  
e1.trabajar(); // Polimorfismo: ejecuta método del hijo
```

Aplicación en automatización:

- Clases base para configuraciones de prueba (**BaseTest**).
- Subclases para pruebas específicas (**LoginTest**, **CompraTest**).

Colecciones en Java

Concepto:

Las colecciones permiten almacenar y manipular grupos de datos dinámicamente.

Tipos comunes:

- **List** → datos ordenados, permite duplicados.
- **Set** → no permite duplicados.
- **Map** → pares clave-valor.

```
List < String > nombres = new ArrayList < >();
nombres.add("Ana");
nombres.add("Juan");

for (String n: nombres) {
    System.out.println(n);
}

Map < String,
String > usuarios = new HashMap < >();
usuarios.put("admin", "1234");
usuarios.put("tester", "abcd");
System.out.println(usuarios.get("tester"));
```

Uso en automatización:

- Guardar datos de prueba (usuarios, productos, mensajes esperados).
- Procesar listas de elementos web (**findElements**).

Expresiones Lambda y Streams

Concepto: Las lambdas permiten escribir código más conciso, especialmente para filtrar o transformar datos.

Ejemplo con List:

```
List < Integer > numeros = Arrays.asList(1, 2, 3, 4, 5);  
  
// Imprimir solo números pares  
numeros.stream().filter(n -> n % 2 == 0).forEach(System.out::println);
```

Ejemplo con Map:

```
Map < String,  
Integer > productos = Map.of("Mouse", 50, "Teclado", 100, "Monitor", 300);  
  
productos.entrySet().stream().filter(p -> p.getValue() > 80).forEach(p -> System.out.println(p.getKey() + " cuesta " + p.getValue()));
```

Uso práctico en testing:

Filtrar datos de una tabla web, validar listas de resultados, o generar subconjuntos de test data

Práctica

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

Ejercicio 1 - Crear una clase `Producto` con atributos `nombre` y `precio`.

Agregar métodos `mostrar()` y `aplicarDescuento(double porcentaje)`.

Ejercicio 2 - Crear una lista de productos y:

- Mostrar todos
- Filtrar los que valen más de \$100
- Aplicar 10% de descuento con una lambda

Ejercicio 3 - Crear una jerarquía `Empleado` → `Tester` → `Automatizador` con métodos sobrescritos.

Ejercicio 4 - Usar un `Map<String, String>` para simular usuarios de login y validar accesos.

Ejercicio 1 - Crear una clase **Producto** con atributos **nombre** y **precio**. Agregar métodos **mostrar()** y **aplicarDescuento(double porcentaje)**.

```
public class Producto {
    private String nombre;
    private double precio;

    public Producto(String nombre, double precio) {
        if (precio < 0) throw new IllegalArgumentException("El precio no puede ser negativo");
        this.nombre = nombre;
        this.precio = precio;
    }

    public void mostrar() {
        System.out.println(toString());
    }

    /**
     * Aplica un descuento porcentual. Ej: 10 -> 10%
     */
    public void aplicarDescuento(double porcentaje) {
        if (porcentaje < 0 || porcentaje > 100) {
            throw new IllegalArgumentException("Porcentaje inválido");
        }
        double factor = 1 - (porcentaje / 100.0);
        this.precio = Math.round((this.precio * factor) * 100.0) / 100.0; // redondeo 2 decimales
    }

    // Getters/setters básicos (encapsulamiento)
    public String getNombre() {
        return nombre;
    }
    public double getPrecio() {
        return precio;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public void setPrecio(double precio) {
        if (precio < 0) throw new IllegalArgumentException("El precio no puede ser negativo");
        this.precio = precio;
    }

    @Override
    public String toString() {
        return "Producto{nombre='" + nombre + "', precio=" + precio + "}";
    }
}
```

```
// Demo rápida
public static void main(String[] args) {
    Producto p = new Producto("Teclado", 150.0);
    p.mostrar();
    p.aplicarDescuento(10); // 10%
    p.mostrar();
}
```

Ejercicio 2 - Crear una lista de productos y:

- Mostrar todos
- Filtrar los que valen más de \$100
- Aplicar 10% de descuento con una lambda

Nota: Reutiliza la clase Producto del ejercicio 1 (mantenela en el mismo paquete/carpeta o importala según corresponda).

```
import java.util. * ;
import java.util.stream.Collectors;

public class GestionProductos {
    public static void main(String[] args) {
        List < Producto > productos = new ArrayList < >();
        productos.add(new Producto("Mouse", 50));
        productos.add(new Producto("Teclado", 120));
        productos.add(new Producto("Monitor", 300));
        productos.add(new Producto("Pad", 20));

        System.out.println("=== Todos los productos ===");
        productos.forEach(Producto::mostrar);

        System.out.println("\n=== Filtrados (> 100) ===");
        List < Producto > caros = productos.stream().filter(p -> p.getPrecio() > 100).collect(Collectors.toList());
        caros.forEach(Producto::mostrar);

        System.out.println("\n=== Aplicando 10% de descuento a todos (lambda + forEach) ===");
        productos.forEach(p -> p.aplicarDescuento(10));
        productos.forEach(Producto::mostrar);
    }
}
```

Ejercicio 3 - Crear una jerarquía `Empleado` → `Tester` → `Automatizador` con métodos sobrescritos.

```
// Empleado.java
public class Empleado {
    private String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }

    public void trabajar() {
        System.out.println(nombre + " está trabajando.");
    }
}
```

```
// Tester.java
public class Tester extends Empleado {
    public Tester(String nombre) {
        super(nombre);
    }

    @Override
    public void trabajar() {
        System.out.println(getNombre() + " está ejecutando pruebas funcionales.");
    }

    public void reportarBug(String titulo) {
        System.out.println(getNombre() + " reporta bug: " + titulo);
    }
}
```

```
// Automatizador.java
public class Automatizador extends Tester {
    public Automatizador(String nombre) {
        super(nombre);
    }

    @Override
    public void trabajar() {
        System.out.println(getNombre() + " está diseñando y manteniendo tests automáticos (Selenium).");
    }

    public void ejecutarSuite() {
        System.out.println(getNombre() + " ejecuta la suite de regresión CI/CD.");
    }
}
```

Ejercicio 3 - Crear una jerarquía **Empleado** → **Tester** → **Automatizador** con métodos sobrescritos.

```
// DemoJerarquia.java
public class DemoJerarquia {
    public static void main(String[] args) {
        Empleado e = new Empleado("Alex");
        Empleado t = new Tester("Brenda");
        Empleado a = new Automatizador("Carlos");

        e.trabajar(); // Alex está trabajando.
        t.trabajar(); // Brenda está ejecutando pruebas funcionales.
        a.trabajar(); // Carlos está diseñando y manteniendo tests automáticos...

        // Downcasting con verificación (para usar métodos específicos)
        if (t instanceof Tester) {
            ((Tester) t).reportarBug("Login falla con credenciales válidas");
        }
        if (a instanceof Automatizador) {
            ((Automatizador) a).ejecutarSuite();
        }
    }
}
```

Ejercicio 4 - Usar un `Map<String, String>` para simular usuarios de login y validar accesos.

Reglas simples:

- `Map` almacena `usuario` -> `password`.
- Función `login(usuario, pass)` retorna `true/false`.
- Se valida existencia y coincidencia.

Extensión natural para automatización: leer el Map desde un properties o CSV y alimentar pruebas de login (data-driven).

```
import java.util.HashMap;
import java.util.Map;

public class LoginSimulado {
    private static final Map<String, String> usuarios = new HashMap<>();

    static {
        // seed de usuarios de prueba
        usuarios.put("admin", "1234");
        usuarios.put("tester", "abcd");
        usuarios.put("guest", "guest");
    }

    public static boolean login(String user, String pass) {
        if (user == null || pass == null) return false;
        String esperado = usuarios.get(user);
        return esperado != null && esperado.equals(pass);
    }

    public static void main(String[] args) {
        System.out.println("Login admin/1234 -> " + (login("admin", "1234") ? "OK" : "FAIL"));
        System.out.println("Login tester/abcd -> " + (login("tester", "abcd") ? "OK" : "FAIL"));
        System.out.println("Login tester/xxxx -> " + (login("tester", "xxxx") ? "OK" : "FAIL"));
        System.out.println("Login inexistente/1234 -> " + (login("foo", "1234") ? "OK" : "FAIL"));
    }
}
```

Cierre

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

Preguntas de repaso

1. ¿Qué ventajas tiene encapsular datos en clases?
2. ¿Cómo se aplica la herencia en frameworks de automatización?
3. ¿Qué diferencia hay entre una `List` y un `Map`?

¿Preguntas?

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

¡Gracias!

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua